2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) | 979-8-3315-0620-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/TrustCom63139.2024.00233

# THEF: A Privacy-Preserving Framework for Transformer Inference leveraging HE and TEE

Zehao Li*†, Jiachun Liao†, Jinhao Yu†, Lei Zhang†
* Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China
lizehao_cs@sjtu.edu.cn
† Big Data Research Center, Nanhu Laboratory, Jiaxing, China
{jliao, yujinhao1996, zl}@nanhulab.ac.cn

***Abstract*—The transformer has profoundly driven the development of natural language processing and computer vision tasks. However, their widespread deployment has raised concerns about the potential leakage of data privacy. To address the issue, homomorphic encryption (HE) is employed as a secure technique to process user private inputs over ciphertexts. As HE only supports linear operations, non-linear functions are required to be approximated into polynomials, which leads to heavy communication and computation overhead. In this paper, we propose THEF (Tee-and-HE-based Framework), an efficient and secure inference framework for transformer inference, which uses HE to ensure data security and leverages Trusted Execution Environment (TEE) to reduce communication and computation cost. We also design a number of optimization methods, including expanding encoding matrix multiplication scheme, batch acceleration and adaptive scheme, to improve performances of the THEF. Finally, we compare the performances of THEF and the state-of-the-art methods. The experimental results demonstrate that THEF maintains a level of accuracy close to that of plaintext model, reduces end-to-end communication cost by 80% and achieves significant improvement in end-to-end run-time.**

***Index Terms*—homomorphic encryption, transformer, privacy-preserving, trusted execution environment**

## I. Introduction

In recent years, transformer-based models, such as GPT [1], have demonstrated remarkable performance in a variety of real-world tasks. Currently, ChatGPT [2] is one of the most widely used commercial general-purpose models globally. As an example of machine learning as a service (MLaaS), the model provider (OpenAI) hosts its own model on CSP (cloud service provider) such as Azure Cloud, AWS, and users need to upload their data to the cloud server to obtain model inference results. However, transformer-based models present a potential vulnerability to data breaches. In the event of a compromise of the cloud server by an adversary, the user's inputs and inference results may be accessed, raising concerns about data privacy.

To guarantee the safety of user data in machine learning (ML) tasks, recent works have proposed some secure inference protocols based on multi-party secure computation (MPC) [5] and homomorphic encryption (HE) [6]. Although MPC-based protocols can be employed for general operations, they can result in significant communication and computing overhead due to their expensive communication primitives. HE can compute encrypted data without decrypting the ciphertext, and obtaining encrypted results that match plaintext calculations. HE also supports non-interactive operations to reduce the significant communication costs compared with MPC. Therefore, HE is considered as an appropriate option for secure inference of transformer-based models.

While HE provides computing flexibility and convenience, it also causes a significant amount of computing overhead. Applying HE to secure transformer inference presents two principal challenges. Firstly, transformer inference consists of lots of large-scale matrix multiplications, necessitating a more efficient implementation for linear functions. Secondly, the calculation of non-linear functions in HE is costly, as the non-linear functions must be approximated into a polynomial that only consists of multiplication and addition operations. Additionally, transformer-based models employ more complex non-linear functions with thousands of inputs, which can significantly increase communication cost [12].

Some works propose the use of TEE (Trusted Execution Environment) to accelerate the HE-based secure inference of traditional ML models [8] [7]. Nevertheless, existing works that combine HE and TEE have not yet addressed the challenge for large-scale matrix multiplications. To address the issue, we propose an expanding encoding scheme to enhance the efficiency of matrix multiplications. Existing works that introduce TEE for acceleration also cause the extra communication cost between the server and TEE and time cost for encryption and decryption. To address the problem, we present batch acceleration, which summits multiple tasks to TEE at once to reduce communication rounds. Additionally, we also implement the adaptive scheme to reduce the overhead of frequent encryption and decryption by selecting appropriate parameters for the HE scheme.

Our contributions are as follows:

1) We propose THEF, an efficient framework using HE for secure inference of transformer and TEE for acceleration. In the framework, expanding encoding scheme is proposed for efficient large-matrix multiplications and several optimization methods are designed to increase the accuracy and efficiency of non-linear functions and reduce the communication and time cost for transformer inferences.

Corresponding author: Lei Zhang. This work was supported by National Key R&D Program of China (2022YFB4501500, 2022YFB4501502).



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

2) We achieve a number of optimizations in THEF, including noise elimination, batch acceleration for less communication cost, adaptive scheme for efficient HE calculation and less overhead of encryption and decryption.
3) We implement THEF using CKKS [15] as HE scheme and Intel SGX [16] as TEE, and evaluate its accuracy, communication cost and run-time performance on BERT-base model of transformers. The results show that THEF achieves the accuracy close to plaintext model results, at least $5\times$ less communication cost and $10\times$ faster run-time on end-to-end performance compared with state-of-the-art methods.

## II. Related Work

### A. Privacy-Preserving Machine Learning combining TEE and HE

In order to enhance the efficiency of HE in private machine learning inference tasks, some works propose utilising TEE to accelerate HE operations. [17] proposes using Intel SGX (A well-known TEE) to reduce approximate calculations in HE to improve performance. [18] proposes a privacy neural network prediction framework that combines HE and TEE, and defines security models at different security levels. [4] proposes a hybrid framework that combines HE with Intel SGX and implements the data-oblivious execution block base on oblivious primitives to prevent leakage of memory access information. Slalom [8] runs Deep Neural Networks (DNNs) in an Intel SGX enclave and selectively divides works to an untrusted GPU.

### B. Private Transformer Inference Frameworks

There are many existing works for secure inference of transformers in different classifications of privacy protocols. 1) HE-based: Iron [11] and Bolt [12]; 2) Secret-Sharing-based: MPCFormer [19], SIGMA [20] 3) Garbled-Circuit-based: Primer [21]. HE-based protocols attract attention due to its good applicability, but it also causes a lot of computation overhead. Thus, we propose THEF framework to accelerate HE operations in secure transformer inference leveraging TEE. To the best of our knowledge, we first introduce TEE to accelerate the HE-based protocols for private transformer inference. Based on TEE, we further design the adaptive scheme that selects appropriate parameters and switch between different computation tasks automatically. THEF provides a more convenient and efficient approach for secure inference of transformers.

## III. Preliminaries

### A. Homomorphic Encryption

Homomorphic encryption represents a crucial cryptographic primitive that permits the calculation of ciphertext, with the ciphertext result matching the plaintext calculation. This makes it an ideal solution for secure outsourcing computing scenarios. In this paper, we use CKKS [15] homomorphic encryption scheme. The primary public parameters of HE schemes include

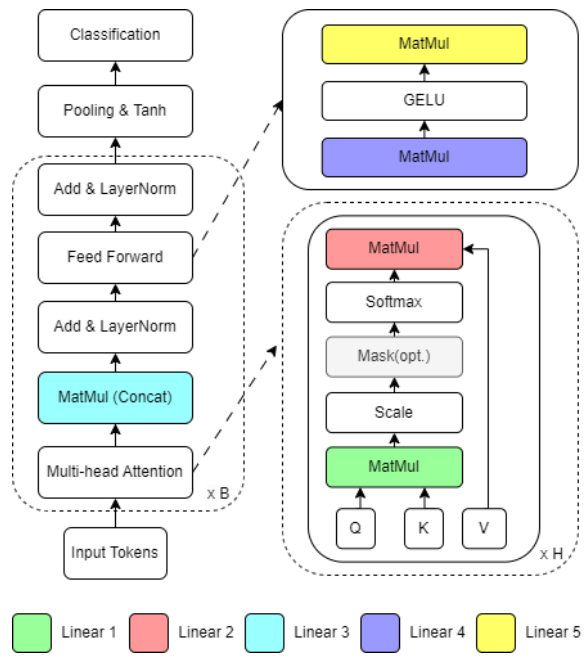


Fig. 1. Transformer inference high-level architecture.

the dimension of the polynomial ring $N$, the plaintext modulus $p$, and the ciphertext modulus $t$. $\kappa$ is the security parameter. The HE scheme typically includes the following algorithms: HE supports efficient linear calculations, including addition *HE.Add* and multiplication *HE.Mult*. For non-linear calculations, HE scheme needs to use polynomial approximation to calculate by the operations start with *HE.Eval*, which will consume the number of multiplications depth *approxDepth*. The higher *approxDepth* signifies a larger degree of the approximation polynomial, which can obtain a more accurate calculation result. However, it will increase the calculation overhead and consume more multiplication depth of ciphertext. *HE.EvalDivide(x)* can calculate $\frac{1}{x}$ with multiplications depth *approxDepth*.

### B. Transformers

While THEF supports various transformer models, we concentrate on BERT (Bidirectional Encoder Representations from Transformers) [23], which is one of the most widely used transformer [22] models. The high-level architecture of the Bert is illustrated in Fig. 1. The transformer tokenizes the user input with input embedding and positional encoding, then processes the data with B consecutive encoder blocks, and finally sends the results to the tanh and classification layer. The functions of the different layers are as follows:

**Multi-head Attention.** Given number of heads $H$ and the parameter of the linear projection $W^O$, Multi-head Attention computes:

$$MultiHead(x) = Concat(head_1, ..., head_H)W^O \quad (1)$$



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

For each attention $head$, given a query $Q$ and weight matrices $K, V$, attention function computes:

$$Attention(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = Softmax(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d}}) \cdot \boldsymbol{V} \quad (2)$$

where the Softmax function computes:

$$Softmax(\boldsymbol{x})[i] = \frac{\exp(\boldsymbol{x}[i])}{\sum_i \exp(\boldsymbol{x}[i])} \quad (3)$$

**Layer Normalization.** For input vector $\boldsymbol{x} \in \mathbb{R}^d$, $\mu = \frac{\sum_i \boldsymbol{x}[i]}{d}$ is the mean of $\boldsymbol{x}$ and $\sigma = \sum_i (\boldsymbol{x}[i] - \mu)^2 \in \mathbb{R}$ is the variance, the result of computing the attention is normalized across the layer:

$$LayerNorm(\boldsymbol{x}) = \frac{\boldsymbol{\gamma} \cdot (\boldsymbol{x} - \mu)}{\sigma} + \boldsymbol{\beta} \quad (4)$$

where $\boldsymbol{\gamma} \in \mathbb{R}^d$ and $\boldsymbol{\beta} \in \mathbb{R}^d$ are two parameters.

**Feed-forward.** The feed-forward network consists of two linear projections and an activation function between them, which computes $FeedForward(\boldsymbol{X}) = W_1 \cdot GELU(W_0 \boldsymbol{X})$. The activation function $GELU$ used in Transformer computes:

$$GELU(x) = \frac{x}{2} \cdot (Erf(\frac{x}{\sqrt{2}}) + 1) \quad (5)$$

where Gauss error function is $Erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$.

**Tanh.** The tanh function is a hyperbolic function used before classification, which computes:

$$Thah(x) = \frac{e^{2x} - 1}{e^{2x} + 1} \quad (6)$$

## IV. Secure Inference Framework for Transformers

### A. Security Analysis

**Security in THEF.** The framework workflow comprises three entities: Client (launch HE task), Server (compute HE operations), TEE (accelerate HE computing). The client generates a public and private key pair to encrypt and decrypt data. The client encrypts input data locally and sends ciphertexts to the server. The server can only receive the encrypted data and learn nothing about the plaintext corresponding to the ciphertext. The server does not have private key, so it cannot decrypt any intermediate results in HE computation. The client sends the private key to TEE via a secure channel. TEE and server send data encrypted with public key to each other. Therefore, the security of THEF (regardless TEE hardware) is equivalent to the security of the CKKS algorithm. The CKKS algorithm is based on cryptographic assumptions proven to be secure. So THEF satisfies security outside TEE hardware.

**Details in TEE security of SGX.** THEF uses Intel SGX as TEE, which is one of the most representative TEEs. Attacks on SGX enclaves are widely studied and included in most kinds of attacks on TEEs [9]. SGX enclaves execute programs in a process space that is completely isolated from the host (including a potentially malicious operation system). Specifically, the memory of enclaves is totally encrypted and authenticated, and its integrity is guaranteed. However, SGX is risk from a range of side-channel attacks. The majority of these attacks are performed by executing data dependent code (e.g. random memory access and branching) in the enclave that can be concurrently observed by other processes running on the same host [3]. Nonetheless, these side-channel vulnerabilities pose a minor threat to our framework because the computations of transformers in THEF are data-oblivious, which apply uniform operations irrespective of the specific input data.

### B. Framework structure and workflow

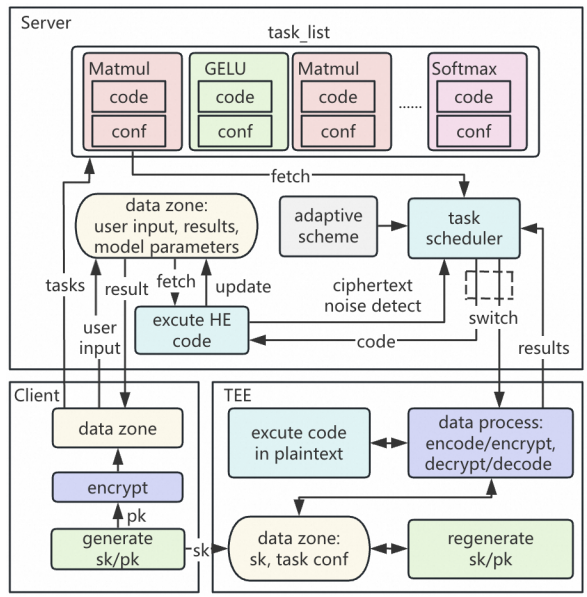


Fig. 2. THEF framework structure and workflow.

As illustrated in Fig.2, the THEF framework comprises three parties: the client, the server, and the TEE. Specifically, the entire private transformer inference process can be divided into multiple tasks, including Matmul(linear), Softmax, GELU, LayerNorm and Tanh functions respectively. Each task consists of execution code related to function and its configuration. THEF executes the first phase of linear functions by HE operations over ciphertexts in the server (§IV-E1), and executes the second phase of linear function, Softmax, GELU, LayerNorm and Tanh over plaintexts in TEE. TEE and the server process the tasks cooperatively through the task scheduler. For the concern about the limitation of executed resource in TEE, we also implement four non-linear functions by HE operations, which supports executing HE code of non-linear functions in the server over ciphertexts. We will introduce more details of the configuration of each task and adaptive scheme in §IV-D1, and specific implementations of functions in §IV-E.

The specific execution process of the framework is as follows: 1) The client generates private and public key pair, then sends the private key to the TEE after remote attestation via secure channel; 2) The client encrypts the input data into ciphertexts with public key, then submits the ciphertexts, public key and the task list of {task_execute_code,



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

task_configuration} pairs to the server; 3) The server encrypts model parameters with public key into ciphertexts (optional), and execute tasks in the task list sequentially; 4) According to the configuration of each executed task, the task scheduler selects to submit its plaintext code, input ciphertexts and configuration of next task to TEE for execution acceleration, or execute the HE code locally; 5) For the tasks submitted to TEE, TEE decrypts the ciphertexts with private key and executes task in plaintext; 6) TEE regenerates public and private key pair and encrypts the results with parameters based on the configuration of next task; 7) TEE stores the new private key locally, and sends the task result ciphertext and new public key back to the server; 8) Repeat steps 3) to 7) until all tasks in the task list are completed, then the server returns the final result ciphertext to the client; 9) The client receives the latest private key from TEE via secure channel, and decrypts the result ciphertext to get the final plaintext result.

### C. Configuration of Task

Each task has its own configuration, which consists of different parameters. Task scheduler will choose the task executed environment (Server/TEE) based on the configuration. If a task is submitted to TEE, the new encryption parameters will depend on the configuration of next task. Each parameter can be set by user manually , while some parameters can be set by adaptive scheme(§IV-D1) automatically. The parameters that can be configured for each task are described as follows:

*1) TEE-enabled:* Enable TEE to accelerate task computing. If the parameter is false, the server will execute the HE code of task locally. Considering the concerns for data security in TEE(§IV-A), the executing resource of TEE and the communication cost between server and TEE, the parameter can be set to false. Optional values: true(default) / false.

*2) Encryption security parameter:* The security level of encryption. The higher security parameter will bring higher the security of the ciphertext, but the size of ciphertext will also be larger. Optional values: 128bit (default) / 192bit / 256bit.

*3) Upper bound of columns (max columns):* The maximum number of plaintexts that can be packed in ciphertext, which needs to be less than half of the ring degree. A higher upper bound of columns means that more plaintext data can be carried in the ciphertext and calculated simultaneously, but the cost of all HE calculations will also increase. Optional values: manual (positive integers).

*4) Ring degree:* The degree of the encryption polynomial used in the CKKS algorithm, which must be larger than double max columns. Optional values: manual (positive integers) / auto.

*5) Multiplication Depth:* The maximum number of multiplications that can be calculated for a fresh ciphertext before the ciphertext noise exceeds the limit. The higher depth can reduce the number of bootstrapping (operation to eliminate noise in ciphertext), but it will also increase the encryption and decryption and HE calculation overhead. Optional values: manual (positive integers) / auto.

We introduce the detailed configuration of each function on the evaluation of THEF. For all non-linear functions, we enable TEE for acceleration. TEE-enabled of linear functions is set to false, for it is efficient to execute multiplication in HE and reduces communication cost between server and TEE. Encryption security parameter of all functions is set default value 128bit. The remaining parameters related to adaptive scheme are demonstrated in §IV-D1.

TABLE I
THE PARAMETERS RELATED TO ADAPTIVE SCHEME OF FUNCTIONS

| Function | Max Columns | Ring Degree | Mul. Depth |
|---|---|---|---|
| Linear (1-5) | 768 | 2048 | 1 |
| Softmax | 768 | 2048 | 18 |
| Tanh | 3072 | 8192 | 16 |
| GELU | 3072 | 8192 | 20 |
| LayerNorm | 768 | 2048 | 12 |

### D. Optimizations

To improve the efficiency and convenience of the framework, we implement some optimizations in THEF. We will present expanding encoding scheme to improve the performance of matrix multiplication in §IV-E1. In addition, adaptive scheme and noise elimination are used to improve the efficiency of executing HE code, and batch acceleration is used to submit multiple tasks to TEE to decrease the communication rounds, encryption and decryption times. Details of these optimizations are shown as follows:

*1) Adaptive scheme:* Adaptive scheme is used to optimize the execution of HE code, which supports selecting appropriate values for both ring degree and multiplication depth based on max columns automatically. On the premise of meeting HE parameter requirements(§IV-C4), less values of both parameters have better calculation performance. For ring degree, THEF will choose the power of 2 that is closest to and greater than upper bound of columns as ring degree when the parameter is auto. Specifically, we set Ring Degree $= 2^m$, where Max Columns $<= 2^{(m-1)}$ and $2^{(m-2)} <$ Max Columns. For multiplication Depth, THEF will calculate the sum of the number of multiplications required in each task as multiplication depth when the parameter is auto.

The specific parameters related to adaptive scheme are shown on Table I. Max columns is set according to the parameter size of each function in Bert. Ring degree and multiplication depth are provided by adaptive scheme according to above rules. The multiplication depth of each function is related to the detailed HE implementation of functions, which is described in §IV-E and §IV-F.

*2) Noise Elimination:* Noise elimination is used to increase the remind remaining multiplication depth in the ciphertext. When the server executes HE code, noise in ciphertext will increase and the number of remaining multiplications will decrease. The ciphertext cannot be decrypted correctly if the noise exceeds the limitation (the number of remaining



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

multiplications less than zero). Using the parameter of multiplication depth provided by adaptive scheme can avoid the noise elimination, but the parameter set by user may still require to execute noise elimination.

Therefore, before executing each homomorphic computation that consumes multiplications, THEF will detect the noise of ciphertext by checking whether the remaining number of multiplications in ciphertext is sufficient. If not, the task scheduler will send the ciphertext to TEE, then TEE decrypts the ciphertext and encrypt it to get fresh ciphertext. Finally, TEE sends the new ciphertext back to the server and the server continues to execute HE computation. Compared with the common approach by bootstrapping [10], THEF can eliminate noise in HE ciphertext automatically and efficiently. More evaluation details of noise elimination are shown in §V-B.

*3) Batch Acceleration:* When multiple consecutive tasks need to be accelerated in TEE, the task scheduler will send these tasks and their ciphertext inputs to TEE simultaneously. TEE executes these tasks in sequence and returns the final encrypted results to the server. Batch acceleration can reduce the number of communication rounds between TEE and the server, but if the executed functions have too much ciphertext inputs, it will cause a lot of encryption and decryption overhead. Therefore, we only use batch acceleration in THEF when executing continuous non-linear functions.

Specifically, we divide the linear function into multiplication and inner product two phases by expanding encoding scheme (§IV-E1). After the multiplication phase executed, we submit the inner product phase and next non-linear task to TEE at once. For the final inference process of LayerNorm and Tanh functions, we send them to TEE for acceleration simultaneously.

### *E. Implementation of transformer functions in THEF*

As illustrated in Fig. 1, despite the multitude of calculation layers and blocks involved in the transformer inference process, they primarily comprise fundamental functions, including MatMul, Softmax, Tanh, GELU and LayerNorm functions. As shown in Fig. 2, task scheduler selects to execute HE code in the server or plaintext code in TEE for each function according to its configuration. Appropriate HE parameters of functions will be set before executed in the server by adaptive scheme(§IV-D1). For the case that the inner product of linear function or non-linear function followed by a non-linear function, we submit them together to TEE by applying batch acceleration(§IV-D3). In this section, we will introduce the detailed implementation of transformer functions in THEF.

*1) Expanding Encoding Matrix Multiplication:* In THEF, we implement the expanding encoding scheme for efficient matrix multiplication. This scheme divides the matrix multiplication into two phases: multiplication and inner product. It encodes the plaintext matrix into a redundant matrix over ciphertext, which allows the multiplication result to be obtained by only one homomorphic multiplication, then calculating inner product after decryption in TEE.

Specifically, we execute multiplication phase in the server. For matrix multiplication $Z = X \cdot Y$, matrices $X$ and $Y$ can be encoded by expanding into one-dimensional vectors. $X$ is $a \times b$ matrix and $Y$ is $b \times c$ matrix. For matrix $X$, each row is repeated $c$ times in row order. For matrix $Y$, repeat $Y$ in column order for $a$ times. The result is

$$\widetilde{X} = \{\underbrace{X_{1:}, ..., X_{1:}}_{c\ times}, \underbrace{X_{2:}, ..., X_{2:}}_{c\ times}, ..., \underbrace{X_{a:}, ..., X_{a:}}_{c\ times}\} \tag{7}$$

$$\widetilde{Y} = \{\underbrace{Y_{:1}, Y_{:2}, ..., Y_{:c}, Y_{:1}, Y_{:2}, ..., Y_{:c}, ..., Y_{:1}, Y_{:2}, ..., Y_{:c}}_{a\ times}\} \tag{8}$$

The server encrypts $\widetilde{X}, \widetilde{Y}$ with homomorphic encryption algorithm, then executes the homomorphic multiplication $\widetilde{Z} = \widetilde{X} \cdot \widetilde{Y}$:

$$\begin{aligned}\widetilde{Z} &= \widetilde{X} \cdot \widetilde{Y} \\ &= \{\underbrace{X_{1:} \cdot Y_{:1}, ..., X_{1:} \cdot Y_{:c}}_{c\ times}, ..., \underbrace{X_{a:} \cdot Y_{:1}, ..., X_{a:} \cdot Y_{:c}}_{c\ times}\}\end{aligned} \tag{9}$$

So far the server completes the homomorphic multiplication phase, then we need to compute the inner product in TEE. The server sends encrypted results of multiplication phase to TEE. TEE decrypts the result ciphertexts $\widetilde{Z}$, then executes inner product phase. We denote the $n$-th element of vector $\widetilde{Z}$ as $Z[n]$. For any $\hat{Z}_{i,j}$, we have:

$$\hat{Z}_{i,j} = \sum_{k=1}^{b} \widetilde{Z}[(i-1) * b * c + (j-1) * b + k] \tag{10}$$

Thus, by summing the adjacent $b$ elements in vector $\widetilde{Z}$, TEE can obtain the element order of the matrix multiplication result $\hat{Z}$.

In the implementation of our framework, matrix multiplications are always followed by non-linear functions that need to be accelerated in TEE. So we submit the inner product calculation and next task to TEE, then calculate inner product before the non-linear task in TEE. By this way, we can compute the inner product in plaintext, avoiding the large overhead of homomorphic computation, and reduce the number of homomorphic multiplications by redundantly encoding the matrix.

*2) Non-linear function:* In THEF, we execute all non-linear functions over plaintexts in TEE, including Softmax, Tanh, GELU, LayerNorm. The plaintext implementation of each function is similar to its common implementation running on plaintext data, as shown in §III-B. The server sends ciphertext inputs and executed code of a batch of tasks to TEE by batch acceleration, while extra encrypted parameter $\gamma$ and $\beta$ also be sent to TEE for LayerNorm function. TEE decrypts ciphertexts, execute tasks in batch order, regenerates private and public key pair. Finally, TEE encrypts the results with parameters of linear functions and send back to the server.

Although we execute plaintext code in TEE for non-linear functions in THEF for evaluation, considering data security



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

concern and the executing resource limitation of TEE, THEF also supports executing HE code of non-linear functions in the server. We will introduce the HE implementation of non-linear functions in the §IV-F.

### F. HE implementations of non-linear functions

Using TEE to accelerate the calculation of non-linear functions will also cause new issues. Increasing the number of functions executed in TEE may raise the concern of exceed the TEE limitation of computational resource. Therefore, we implement the HE implementations of non-linear functions, which support executing non-linear functions with HE operations in the server. For all non-linear functions in THEF, we choose *approxDepth* = 8 (§III-A).

*1) Softmax:* Softmax (cf. Eq. 3) requires evaluating the exponential and division function, and the inner product of all columns in the ciphertext. The HE scheme supports calculating the inner product by function *HE.InnerSum*. The HE implementation of Softmax is shown in Algorithm 1.

**Algorithm 1** The HE implementation for Softmax.

**Input:** The ciphertext input $X$.
1: exp_x = HE.EvalFunc(exp, X, approxDepth)
2: sum_exp_x = HE.InnerSum(exp_x)
3: div_sum_exp_x = HE.EvalDivide(sum_exp_x)
**return:** HE.Mult(exp_x, div_sum_exp_x)

*2) Tanh:* Tanh is a hyperbolic function commonly used in neural networks, which evaluates $Tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}$. It also requires evaluating the exponential and division function. The HE implementation of Tanh is shown in Algorithm 2.

**Algorithm 2** The HE implementation for Tanh.

**Input:** The ciphertext input $X$.
1: double_x = HE.Mult(X, 2)
2: exp_2x = HE.EvalFunc(exp, double_x)
3: exp_2x_a1 = HE.Add(exp_2x, 1)
4: exp_2x_s1 = HE.Sub(exp_2x, 1)
5: div_exp_2x_a1 = HE.EvalDivide(exp_2x_a1)
**return:** HE.Mult(exp_2x_s1, div_exp_2x_a1)

*3) GELU:* Iron [11] proposes the accurate polynomial approximation for GELU using the following equation:

$$GELU(x) = \frac{x}{2}(1 + Tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3))) \quad (11)$$

We evaluate GELU with the above equation and the Tanh function in §IV-F2. The HE implementation of GELU for HE-case is shown in Algorithm 3.

*4) LayerNorm:* LayerNorm (cf. Eq. 4) requires to calculate division function and the inner product of columns in the ciphertext. Unlike other non-linear functions that only require input data, LayerNorm also requires two model parameters $\gamma$ and $\beta$. So task scheduler will encrypt $\gamma$ and $\beta$ as the the input of the HE executed code. The HE implementation of LayerNorm is shown in Algorithm 4.

**Algorithm 3** The HE implementation for GELU.

**Input:** The ciphertext input $X$.
1: square_x = HE.Mult(X, X)
2: cubic_x = HE.Mult(X, square_x)
3: res = HE.Mult(cubic_x, 0.044715)
4: res = HE.Add(res, X)
5: res = HE.Mult(res, $\sqrt{2/\pi}$)
6: res = Tanh(res) ▷ §IV-F2
7: res = HE.Add(res, 1)
**return:** HE.Mult(res, X/2)

**Algorithm 4** The HE implementation for LayerNorm.

**Input:** The ciphertext input $X$, ciphertext model parameters $\beta$, $\gamma$.
1: sum_x = HE.InnerSum(X)
2: diff_x = HE.Sub(X - sum_x)
3: sigma_x = HE.Mult(diff_x, diff_x)
4: div_sigma_x = HE.EvalDivide(sigma_x)
5: res = HE.Mult(div_sigma_x, diff_x)
6: res = HE.Mult(res, $\gamma$)
**return:** HE.add(res, $\beta$)

## V. Experiments

In this section, we first introduce the setup of our experiments. Then we evaluate noise elimination performance comparing to traditional approach bootstrapping. To the best of we know, Bolt [12] is the state-of-the-art system of secure transformer inference, and Iron [11] is well-known system that investigates how to keep data privacy in transformer inference. And we conduct the experiments on accuracy, communication cost and run-times for different functions and end-to-end process, comparing with Bolt and Iron.

### A. Experiment Setup

*1) Implement Details:* In the experiments, we use CKKS algorithm provided by OpenFHE open-source library as HE scheme. The OpenFHE library supports to generate the relinearization key for ciphertext multiplications and bootstrap key for noise elimination by its APIs. The configuration we used in the experiment for funtions in THEF is shown on Table I. For all functions in THEF, we set the configuration of the encryption security parameter to 128bit.

*2) Dataset:* To demonstrate the accuracy and efficiency of our framework, we conduct an experiment on the datasets from the GLUE benchmark [13], which is widely used to test the performance of NLP models. The datasets included in the experiment were RTE, STS-B, SST-2 and MRPC. We evaluate the accuracy, communication cost and run-time performance of linear and non-linear functions and compare THEF with the HE-based models Iron [11], Bolt [12] with the data from the above datasets.

*3) Environment Setting:* The experiments are conducted on a TEE-enabled computer (equipped with Intel SGX support), which contains an Intel(R) Xeon(R) Gold 5318S CPU



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

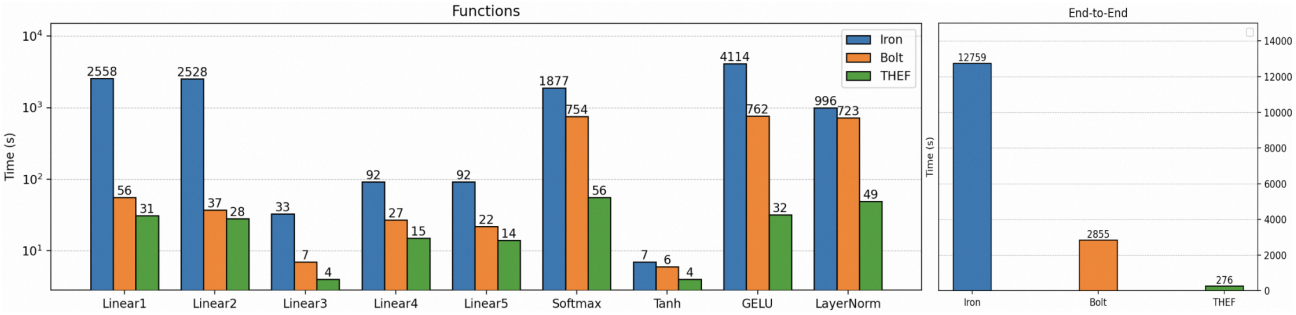


Fig. 3. Run-times of functions and end-to-end on Iron, Bolt and THEF

2.10GHz with 96 cores and 512GB memory. The operating system is Ubuntu 20.04.6 LTS.

### B. Noise elimination

TABLE II
HE CIPHERTEXT NOISE ELIMINATION PERFORMANCE

| Ring degree | | Pre-computation | Time |
|---|---|---|---|
| 8192 | Bootstrapping | 7.73s | 9.92s |
| | THEF | / | 0.017s (583×) |
| 16,384 | Bootstrapping | 35.45s | 17.20s |
| | THEF | / | 0.036s (478×) |
| 32,768 | Bootstrapping | 78.66s | 26.74s |
| | THEF | / | 0.081s (330×) |
| 65,536 | Bootstrapping | 154.22s | 47.39s |
| | THEF | / | 0.148s (320×) |

Ciphertext multiplication will increase the noise in ciphertext and consume the multiplication depths in HE scheme. Bootstrapping [10] is commonly used to decrease the noise and recovery the multiplication depths of ciphertext in HE scheme, but it will cause a lot of computational overhead. In THEF, we replace the bootstrapping with the decryption and re-encryption process in the TEE. Noise elimination can be done automatically by THEF without manually executing bootstrapping, and efficiency is greatly improved. As shown in table II, the bootstrapping process requires a significant amount of pre-computation time. In the ring degree of 65,536, the pre-computation time exceeds 150 seconds. In comparison to the bootstrapping approach, THEF is 583× faster in 8192 ring degree, 478× faster in 16384 ring degree, and more than 320× faster in higher ring degrees. THEF achieves a notable enhancement in the efficiency of both the pre-computation and execution phases for noise elimination.

### C. Accuracy

We compare the accuracy of floating-point results, Iron, Bolt and THEF in Table III. Similar to previous works, we choose the accuracy for RTE, the F1 score for MRPC, the accuracy for SST-2 and the Pearson correlation for STS-B as evaluation results. The plaintext results in the second column are provided by [14]. On four datasets, the accuracy of THEF is close to the plaintext model results, with the maximum accuracy loss of 0.28% on the STS-B dataset. We also achieve better accuracy on most items compared with Bolt and Iron, with the maximum accuracy improvement of 1.68% to Bolt on STS-B dataset. For non-linear functions, Bolt and Iron use polynomials to approximate, which loses some precision. We execute non-linear functions in plaintext in TEE to achieve higher efficiency and accuracy. And homomorphic computation of linear layers causes less accuracy loss, so THEF can achieve precision close to plaintext model approach.

TABLE III
ACCURACY OF FOUR MODELS

| Dataset | Plaintext | Iron | Bolt | THEF |
|---|---|---|---|---|
| RTE | 71.20 | 70.76 | 69.68 | 71.15 |
| MRPC | 90.23 | 89.87 | 90.53 | 90.49 |
| SST-2 | 92.95 | 92.77 | 91.74 | 92.82 |
| STS-B | 89.93 | 89.41 | 87.97 | 89.65 |

### D. Communication Analysis

TABLE IV
COMMUNICATION OVERHEAD COMPARING WITH IRON AND BOLT

| Function | Comm. (MB) | | |
|---|---|---|---|
| | Iron | Bolt | THEF |
| Linear 1 | 4844.14 | 7.06 | 83.61 |
| Linear 2 | 4918.38 | 9.88 | 102.26 |
| Linear 3 | 47.65 | 4.51 | 61.92 |
| Linear 4 | 95.40 | 9.01 | 58.15 |
| Linear 5 | 95.21 | 13.52 | 64.88 |
| Softmax | 3596.32 | 1447.65 | 157.34 |
| Tanh | 20.67 | 16.64 | 11.82 |
| GELU | 7960.00 | 1471.67 | 178.73 |
| LayerNorm | 871.46 | 599.40 | 109.44 |
| **End-to-End** | 280.99 GB | 59.61 GB | 11.76 GB |

Table IV shows the communication cost of different functions in a encoder block and end-to-end in consecutive $B = 12$ encoder blocks. For linear functions, we achieve 58× and 48× less communication on Linear 1 and Linear 2 comparing with Iron. Since Bolt proposes and applies a more efficient



Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.

compact encoding ciphertext multiplication, and the expanding encoding scheme used in THEF improves performance while increasing the number of ciphertexts, Bolt achieves less communication cost on linear functions. For non-linear functions, THEF has less communication cost up to $44\times$ on GELU compared with Iron, and up tp $9\times$ on Softmax compared with Bolt. Through efficient calculation of non-linear functions based on TEE acceleration, THEF achieves $24\times$ and $5\times$ less communication cost in total end-to-end process comparing with Iron and Bolt respectively.

### *E. Time Cost*

We present the run-time results of different functions and end-to-end on Fig. 3. The network setting of bandwidth and latency are 100 Mbps, 40 ms respectively. THEF achieves less run-times in all functions comparing with Iron and Bolt, especially for non-linear functions. Specifically, THEF is $90\times$ faster than Iron on Linear 2 and $1.7\times$ faster than Bolt on Linear 3. For non-linear functions, we achieve significant improvement by TEE acceleration. THEF is $128\times$ faster than Iron and $24\times$ faster than Bolt on GELU. For the entire end-to-end process, we achieve $46\times$ faster than Iron and $10\times$ than Bolt. Despite the improment under certain experiment setting, THEF still significantly outperforms the state-of-the-art approaches for secure transformer inference based on HE protocols.

## VI. Conclusion

We propose THEF, a privacy-preserving and efficient framework based on TEE and HE for transformer inference. THEF offers a practical approach to protect the safety of user data during transformer inference. In THEF, the implementation of transformers encompasses several tasks, along with their respective configurations and the workflow for conducting transformer inference. We design the expanding encoding scheme for efficient large-scale matrix multiplication and also several other optimizations, including adaptive scheme, noise elimination and batch acceleration, to improve the performance of the THEF. Experimental results demonstrate that the THEF achieves better accuracy, less communication cost and less run-times compared to the state-of-the-art methods.

## References


[1] Tom B. Brown, et al. 2020. Language models are few-shot learners. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 159, 1877–1901.

[2] OpenAI ChatGPT. https://openai.com/blog/chatgpt.

[3] Victor Costan, Ilia Lebedev, and Srinivas Devadas. 2016. Sanctum: minimal hardware extensions for strong software isolation. In Proceedings of the 25th USENIX Conference on Security Symposium (SEC'16). USENIX Association, USA, 857–874.

[4] Qifan Wang, et al. 2024. HT2ML: An efficient hybrid framework for privacy-preserving Machine Learning using HE and TEE. Comput. Secur. 135, C (Dec 2023).

[5] Y. Wang, et al., "Characterization of MPC-based Private Inference for Transformer-based Models," 2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Singapore, Singapore, 2022, pp. 187-197.

[6] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. 2019. Efficient Multi-Key Homomorphic Encryption with Packed Ciphertexts with Application to Oblivious Neural Network Inference. Conference on Computer and Communications Security (CCS '19). Association for Computing Machinery, New York, NY, USA, 395–412.

[7] Hanieh Hashemi, et al. 2021. DarKnight: An Accelerated Framework for Privacy and Integrity Preserving Deep Learning Using Trusted Hardware. In MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21). Association for Computing Machinery, New York, NY, USA, 212–224.

[8] Florian Tramèr, Dan Boneh. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019.

[9] M. Sabt, M. Achemlal and A. Bouabdallah, "Trusted Execution Environment: What It is, and What It is Not," 2015 IEEE Trustcom/BigDataSE/ISPA, Helsinki, Finland, 2015, pp. 57-64.

[10] Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y. (2018). Bootstrapping for Approximate Homomorphic Encryption. In: Nielsen, J., Rijmen, V. (eds) Advances in Cryptology – EUROCRYPT 2018. EUROCRYPT 2018. Lecture Notes in Computer Science(), vol 10820. Springer, Cham.

[11] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2024. Iron: private inference on transformers. In Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1143, 15718–15731.

[12] Q. Pang, J. Zhu, H. Möllering, W. Zheng and T. Schneider, "BOLT: Privacy-Preserving, Accurate and Efficient Inference for Transformers," in 2024 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2024 pp. 133-133.

[13] Alex Wang, et al. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP, pages 353–355, Brussels, Belgium. Association for Computational Linguistics.

[14] O. Zafrir, G. Boudoukh, P. Izsak and M. Wasserblat, "Q8BERT: Quantized 8Bit BERT," in 2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS), Vancouver, BC, Canada, 2019 pp. 36-39.

[15] Cheon, J.H., Kim, A., Kim, M., Song, Y. (2017). Homomorphic Encryption for Arithmetic of Approximate Numbers. In: Takagi, T., Peyrin, T. (eds) Advances in Cryptology – ASIACRYPT 2017. ASIACRYPT 2017. Lecture Notes in Computer Science(), vol 10624. Springer, Cham.

[16] Intel Software Guard Extensions. https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html

[17] H. Xiao, Q. Zhang, Q. Pei and W. Shi, "Privacy-Preserving Neural Network Inference Framework via Homomorphic Encryption and SGX," 2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS), DC, USA, 2021, pp. 751-761.

[18] X. Liu, et al. "Efficient and Privacy-Preserving Neural Network Prediction Scheme with TEE," ICC 2023 - IEEE International Conference on Communications, Rome, Italy, 2023, pp. 2306-2311.

[19] Li, Dacheng et al. "MPCFormer: fast, performant and private Transformer inference with MPC." 11th International Conference on Learning Representations, ICLR 2023.

[20] Kanav Gupta, et al. "SIGMA: Secure GPT Inference with Function Secret Sharing," 24th Privacy Enhancing Technologies Symposium (PETS 2024).

[21] Zheng, Mengxin et al. "Primer: Fast Private Transformer Inference on Encrypted Data." 2023 60th ACM/IEEE Design Automation Conference (DAC) (2023): 1-6.

[22] Ashish Vaswani, et al. 2017. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

[23] Jacob Devlin, et al. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.




Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on March 23,2026 at 12:09:25 UTC from IEEE Xplore. Restrictions apply.